ACS Architecture v1.1 (Hardened) — Full Spec + Code Skeleton
Aperiodicity Certificate System (Markers → Deflation → Invariants)

Kevin T.N.
Independent Researcher
jkdkr2439@gmail.com

February 2026

Abstract

This document is a complete architecture specification for the **Aperiodicity Certificate System (ACS)**: a correctness-first pipeline that proposes and verifies certificates of aperiodicity for 2D SFT / Wang tilings using *Markers* M, *Deflation* Φ, and optional *Invariant* $\mathcal{I}$. The design is hardened against failure modes revealed by chaos-style audits: window/boundary artifacts, solver nondeterminism, canonicalization mismatches, prototype coverage gaps, and data lineage poisoning. The second half contains a full Python source skeleton implementing the layered structure, data contracts, tri-state verifier, obligations O1/O2/O3/O3b, and SAT-runner abstraction.

Contents

1 0. Design Goals and Non-Negotiables **3**
1.1 0.1 Goals . . . 3
1.2 0.2 Hard Locks . . . 3

2 1. System Overview **3**

3 2. Layered Architecture (IPOD + DATA) **4**
3.1 2.1 Responsibilities . . . 4
3.2 2.2 Dependency Locks . . . 4

4 3. Risk Register (Chaos-Audit) and Hardening Fixes **4**

5 4. Data Contracts (Schemas) **5**
5.1 4.1 RuleSpec (Input) . . . 5
5.2 4.2 NormalizedRules (Post-canonicalization) . . . 5
5.3 4.3 Certificate (Primary Output) . . . 5
5.4 4.4 VerifierReport (Tri-State) . . . 5

6 5. Verification Obligations (v1.1) **6**
6.1 5.1 Window Policy (Mandatory) . . . 6
6.2 5.2 O1 Marker Soundness (+ Sanity) . . . 6
6.3 5.3 O2 Unique Phase . . . 6
6.4 5.4 O3 Deflation Consistency . . . 6

6.5 5.5 O3b Prototype Coverage (Mandatory) . . . 6
6.6 5.6 O4 Optional Invariants . . . 6

7 6. Deterministic SAT Runner and Verdict Policy **7**
7.1 6.1 Determinism . . . 7
7.2 6.2 Verdict Policy . . . 7

8 7. DATA Layer: Lineage and Anti-Poisoning **7**
8.1 7.1 Keying . . . 7
8.2 7.2 Stores . . . 7

9 8. PROCESS: Proposers, CEGIS, Curriculum **7**
9.1 8.1 Dual Proposer Baseline . . . 7
9.2 8.2 Curriculum on L . . . 7
9.3 8.3 Stability Metrics . . . 8

10 9. Folder Structure (Complete) and File Map **8**

11 10. Full Python Code Skeleton (v1.1) **9**
11.1 10.1 `acs/core/types.py` . . . 9
11.2 10.2 `acs/core/hashing.py` . . . 11
11.3 10.3 `acs/input/rulespec`$_p arser.py$ . . . 11
11.4 10.4 `acs/input/adjacency.py` . . . 11
11.5 10.5 `acs/input/canonicalize.py` . . . 12
11.6 10.6 `acs/input/cnf`$_t emplates.py(Skeleton)$ . . . 12
11.7 10.7 `acs/process/window`$_p olicy.py$ . . . 13
11.8 10.8 `acs/process/obligations.py` . . . 13
11.9 10.9 `acs/output/sat`$_r unner.py$ . . . 14
11.1010.10 `acs/output/verifier.py` . . . 15
11.1110.11 `acs/process/proposer`$_h euristic.py$ . . . 17
11.1210.12 `acs/process/cegis.py` . . . 17
11.1310.13 `acs/cli.py` . . . 18

12 11. Notes on Completing the Skeleton (Implementation Checklist) **18**

13 References **19**

1 0. Design Goals and Non-Negotiables

1.1 0.1 Goals

- **Correctness-first:** no false PASS. If inconclusive, output UNKNOWN.
- **Reproducibility:** same inputs and budget produce identical verifier results.
- **Extensibility:** pluggable solver backend, proposer, invariants, and domain encoders.
- **Separation of concerns:** strict IPOD+DATA layering to avoid dependency tangles.

1.2 0.2 Hard Locks

- **Tri-state verdict:** PASS/FAIL/UNKNOWN. Any timeout/crash yields UNKNOWNand blocks PASS.
- **Phase/window robustness:** O1/O2 must be checked across *multiple windows and boundary modes*.
- **Prototype coverage:** O3b is mandatory; coverage gaps must be ruled out by SAT (or yield UNKNOWN).
- **Hash lineage:** every artifact must carry `rules_hash`; stores are keyed by `rules_hash`.
- **Deterministic solver mode:** pinned solver version/flags and fixed seed in config.

2 1. System Overview

ACS solves three problems:

1. **Canonicalize rules** (Wang/SFT) into a normalized, hashed rule model.
2. **Propose certificates** Cert $= (M, \Phi, \mathcal{I})$ via heuristic or ML proposer.
3. **Verify certificates** with obligations:
   - O1: Marker soundness (misphased interior marker occurrences are impossible) — SAT *UNSAT required*.
   - O2: Unique phase/composition (no patch supports two phases) — SAT *UNSAT required*.
   - O3: Deflation consistency (prototypes legal, adjacency consistent) — finite checks (and optional SAT if wildcarded).
   - O3b: Prototype coverage (no legal $k \times k$ block exists outside prototype set) — SAT *UNSAT required*.
   - O4: Optional invariants (curl-free, drift identities, etc.) — finite checks / optional SAT.

Verifier result is tri-state: PASSif all mandatory obligations pass; FAILif any mandatory obligation finds a witness; UNKNOWNif any mandatory obligation is inconclusive.

3 2. Layered Architecture (IPOD + DATA)

3.1 2.1 Responsibilities

INPUT	Parse/validate/canonicalize rules; build normalized model; build CNF templates (no solving).
PROCESS	Propose certificates; generate obligations; orchestrate CEGIS; choose windows/boundaries; request solver runs.
OUTPUT	Side effects: SAT calls, extraction of witnesses, artifact writing; aggregate tri-state report.
DISPLAY	Pure presentation and formatting; never calls solver.
DATA	Persistent stores: patches, counterexamples, certificate zoo, run registry; strict lineage keying.

3.2 2.2 Dependency Locks

- PROCESS must not import DISPLAY.
- DISPLAY must not import solver code.
- OUTPUT must not call proposer logic; it consumes obligation specs only.
- DATA is passive: no outward calls.

4 3. Risk Register (Chaos-Audit) and Hardening Fixes

Risk	Symptom	Fix (v1.1)
H1 Window instability	O2 passes at small L but fails at larger L; phase flips across placements	Multi-window & multi-boundary verification; phase confidence in report
H2 Solver nondeterminism	Same inputs produce different results; irreproducible witnesses	Deterministic solver flags/seed/version; log CNF hash; timeout/crash → UNKNOWN
H3 Canonical mismatch	bogus boundary mismatches; inflate fails	Stable canonical ordering; store map tables; enforce `rules_hash` stamps
H4 False UNSAT by boundary artifact	O1/O2 vacuously pass due to encoding boundary effects	Interior margin; boundary-mode dual checks; satisfiable sanity queries
H5 Prototype coverage gap	O3 passes but some legal $k \times k$ blocks not in prototypes	New mandatory obligation O3b coverage (SAT search outside prototypes must be UNSAT)
H6 ML overfit/-mode collapse	avoids known counterexamples but does not converge	Always keep heuristic baseline; curriculum on L; novelty/diversity scoring
H7 Data poisoning	samples reused across different rules	Stores keyed by $(rules_hash, k, \ldots)$; hard-fail mismatch

5 4. Data Contracts (Schemas)

5.1 4.1 RuleSpec (Input)

```
{
  "spec_version": "1.0",
  "domain": "wang",
  "alphabet": {
    "tiles": [
      {"id":"a0","N":"c1","E":"c2","S":"c3","W":"c4"}
    ],
    "colors": ["c1","c2","c3","c4"]
  },
  "constraints": {"type":"edge_match", "radius": 1},
  "symmetry": {"allow_rotations": false, "allow_reflections": false},
  "metadata": {"name":"example", "source":"manual|paper|generated"}
}
```

5.2 4.2 NormalizedRules (Post-canonicalization)

Properties:

- Dense integer IDs for tiles/colors (stable ordering).
- Compatibility tables `east_ok`, `north_ok`.
- Deterministic `rules_hash` derived from canonical data.

5.3 4.3 Certificate (Primary Output)

Includes:

- k, marker radius, marker patches, intended phase.
- macro prototypes (each is a $k \times k$ base-tile block) and macro adjacency constraints.
- iterability flag/metadata.
- optional invariant spec.
- provenance (run_id, seed, proposer type).

5.4 4.4 VerifierReport (Tri-State)

```
{
  "result": "PASS|FAIL|UNKNOWN",
  "obligations": {
    "O1_marker_soundness": {"status":"PASS|FAIL|UNKNOWN", "sat_calls": 0},
    "O2_unique_phase": {"status":"PASS|FAIL|UNKNOWN", "sat_calls": 0},
    "O3_deflation_consistency": {"status":"PASS|FAIL", "checks": 0},
    "O3b_prototype_coverage": {"status":"PASS|FAIL|UNKNOWN", "sat_calls": 0},
    "O4_invariant": {"status":"SKIP|PASS|FAIL|UNKNOWN", "checks": 0}
  },
```

```
  "stability": {
    "multi_window": {"windows_tested": 6, "boundaries_tested": 2},
    "phase_confidence": 0.98
  }
}
```


Tri-state lock. Global result is PASSiff all mandatory obligations are PASSand no mandatory obligation is UNKNOWN.

6 5. Verification Obligations (v1.1)

6.1 5.1 Window Policy (Mandatory)

To prevent boundary/window artifacts, O1/O2/O3b must run under:

- Multiple window sizes L.
- Multiple placements (shifts) of the window origin.
- Multiple boundary modes (at least `free` and `periodic`).
- Interior margin $m \geq 3r$ so marker placement cannot be "helped" by window boundary.

6.2 5.2 O1 Marker Soundness (+ Sanity)

O1 requires for misphased interior marker placement. Additionally, we run a satisfiable sanity query to ensure O1 is not vacuously passing due to over-constraining.

6.3 5.3 O2 Unique Phase

O2 requires for the existence of a patch admitting two distinct phases consistent with marker logic.

6.4 5.4 O3 Deflation Consistency

Finite checks ensure prototypes are legal $k \times k$ blocks and macro adjacency implies base-level boundary match. If prototypes include wildcards, SAT fills them; any timeout yields UNKNOWN.

6.5 5.5 O3b Prototype Coverage (Mandatory)

SAT query searches for any legal $k \times k$ block not equal to any prototype. Must be . This blocks coverage gaps.

6.6 5.6 O4 Optional Invariants

Curl-free constraints and drift/scaling identities checked over local squares/prototypes.

7 6. Deterministic SAT Runner and Verdict Policy

7.1 6.1 Determinism

SAT execution is deterministic by contract:

- pinned solver and version,
- fixed seed and fixed flags,
- CNF hash logged,
- incremental SAT is opt-in and must reset state per obligation family.

7.2 6.2 Verdict Policy

- Timeouts/crashes yield obligation status UNKNOWN.
- Any UNKNOWNblocks global PASS.
- Reports separate FAIL(witness exists) from UNKNOWN(inconclusive).

8 7. DATA Layer: Lineage and Anti-Poisoning

8.1 7.1 Keying

All stored objects are keyed by:

$$(\texttt{rules_hash}, k, \texttt{sampler_mode}, \texttt{seed}, \texttt{window_policy_id})$$

Mismatch triggers a hard failure.

8.2 7.2 Stores

- PatchBank: legal patches + provenance.
- CounterexampleBank: SAT witnesses tagged by obligation and boundary/window mode.
- CertificateZoo: candidates + novelty and stability scores.
- RunRegistry: manifests for each run including hashes and tool versions.

9 8. PROCESS: Proposers, CEGIS, Curriculum

9.1 8.1 Dual Proposer Baseline

Heuristic proposer baseline is always available; ML proposer is optional and can propose deltas.

9.2 8.2 Curriculum on L

Start small to find structure; increase window sizes only after stability is high; publish only when target policy passes.

9.3 8.3 Stability Metrics

Verifier returns:

- multi-window counts,
- boundary modes tested,
- phase confidence (fraction of windows agreeing on inferred phase).

10 9. Folder Structure (Complete) and File Map

```
acs/
  __init__.py
  cli.py

  core/
    types.py
    hashing.py
    config.py
    errors.py

  input/
    rulespec_parser.py
    canonicalize.py
    adjacency.py
    cnf_templates.py

  process/
    proposer_base.py
    proposer_heuristic.py
    proposer_ml_stub.py
    obligations.py
    cegis.py
    window_policy.py

  output/
    sat_runner.py
    verifier.py
    artifact_writer.py

  data/
    stores.py
    lineage.py

  display/
    report_md.py

tests/
  test_canonicalize.py
  test_obligations_smoke.py
  test_verifier_tristate.py
```

11 10. Full Python Code Skeleton (v1.1)

This section includes the complete skeleton source referenced by the architecture.

11.1 10.1 acs/core/types.py

```
from __future__ import annotations
from dataclasses import dataclass, field
from enum import Enum
from typing import Dict, List, Tuple, Optional, Any

class TriState(str, Enum):
    PASS = "PASS"
    FAIL = "FAIL"
    UNKNOWN = "UNKNOWN"

@dataclass(frozen=True)
class RuleSpecTile:
    id: str
    N: str
    E: str
    S: str
    W: str

@dataclass(frozen=True)
class RuleSpec:
    spec_version: str
    domain: str # "wang" | "sft_generic"
    tiles: List[RuleSpecTile]
    colors: List[str]
    allow_rotations: bool = False
    allow_reflections: bool = False
    metadata: Dict[str, Any] = field(default_factory=dict)

@dataclass(frozen=True)
class NormalizedRules:
    alphabet_size: int
    color_size: int
    tile_int: Dict[str, int]
    color_int: Dict[str, int]
    # adjacency compatibility: east_ok[a][b] indicates E(a)==W(b)
    east_ok: List[List[bool]]
    north_ok: List[List[bool]]
    rules_hash: str

@dataclass
class MarkerPatch:
    id: str
    grid: List[List[int]] # tiles as ints

@dataclass
class Markers:
    radius: int
```

```
    patches: List[MarkerPatch]
    intended_phase: Tuple[int, int] # (x_mod_k, y_mod_k)

@dataclass
class Deflation:
    macro_prototypes: Dict[str, List[List[int]]] # name -> kxk block of tile ints
    macro_adjacency_east: List[Tuple[str, str]]
    macro_adjacency_north: List[Tuple[str, str]]
    iterable: bool = True

@dataclass
class Invariant:
    type: str # "none" | "height"
    params: Dict[str, Any] = field(default_factory=dict)

@dataclass
class Certificate:
    cert_version: str
    rules_hash: str
    k: int
    markers: Markers
    deflation: Deflation
    invariant: Invariant
    provenance: Dict[str, Any] = field(default_factory=dict)

@dataclass
class SatCallMeta:
    cnf_hash: str
    solver: str
    solver_version: str
    seed: int
    timeout_s: float

@dataclass
class SatResult:
    status: TriState # PASS/FAIL/UNKNOWN as runner-level status
    sat: Optional[bool] = None
    witness: Optional[Dict[str, Any]] = None
    meta: Optional[SatCallMeta] = None
    error: Optional[str] = None

@dataclass
class ObligationResult:
    name: str
    status: TriState
    sat_calls: int = 0
    checks: int = 0
    notes: Dict[str, Any] = field(default_factory=dict)
    counterexample: Optional[Dict[str, Any]] = None

@dataclass
class VerifierReport:
    run_id: str
    rules_hash: str
```

```
    cert_hash: str
    result: TriState
    obligations: Dict[str, ObligationResult]
    stability: Dict[str, Any] = field(default_factory=dict)
    environment: Dict[str, Any] = field(default_factory=dict)
```

11.2 10.2 acs/core/hashing.py

```
import hashlib
import json
from typing import Any

def stable_json(obj: Any) -> str:
    return json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=False)

def sha256_str(s: str) -> str:
    return "sha256:" + hashlib.sha256(s.encode("utf-8")).hexdigest()

def hash_dict(d: Any) -> str:
    return sha256_str(stable_json(d))
```

11.3 10.3 acs/input/rulespec$_parser.py$

```
import json
from acs.core.types import RuleSpec, RuleSpecTile

def load_rulespec(path: str) -> RuleSpec:
    with open(path, "r", encoding="utf-8") as f:
        raw = json.load(f)
    tiles = [RuleSpecTile(**t) for t in raw["alphabet"]["tiles"]]
    colors = raw["alphabet"]["colors"]
    sym = raw.get("symmetry", {})
    return RuleSpec(
        spec_version=raw.get("spec_version", "1.0"),
        domain=raw.get("domain", "wang"),
        tiles=tiles,
        colors=colors,
        allow_rotations=bool(sym.get("allow_rotations", False)),
        allow_reflections=bool(sym.get("allow_reflections", False)),
        metadata=raw.get("metadata", {}),
    )
```

11.4 10.4 acs/input/adjacency.py

```
from typing import List, Tuple
from acs.core.types import RuleSpecTile

def build_adjacency(tiles: List[RuleSpecTile]) -> Tuple[List[List[bool]], List[List[bool
    ]]]:
    n = len(tiles)
```

```
    east_ok = [[False]*n for _ in range(n)]
    north_ok = [[False]*n for _ in range(n)]
    for i,a in enumerate(tiles):
        for j,b in enumerate(tiles):
            east_ok[i][j] = (a.E == b.W)
            north_ok[i][j] = (a.N == b.S)
    return east_ok, north_ok
```

11.5 10.5 acs/input/canonicalize.py

```
from acs.core.types import RuleSpec, NormalizedRules
from acs.core.hashing import hash_dict
from acs.input.adjacency import build_adjacency

def canonicalize(spec: RuleSpec) -> NormalizedRules:
    # stable ordering (fixes canonical mismatch class)
    tiles_sorted = sorted(spec.tiles, key=lambda t: t.id)
    colors_sorted = sorted(set(spec.colors))

    tile_int = {t.id: i for i, t in enumerate(tiles_sorted)}
    color_int = {c: i for i, c in enumerate(colors_sorted)}

    east_ok, north_ok = build_adjacency(tiles_sorted)

    rules_hash = hash_dict({
        "domain": spec.domain,
        "tiles": [t.__dict__ for t in tiles_sorted],
        "colors": colors_sorted,
        "sym": {"rot": spec.allow_rotations, "ref": spec.allow_reflections},
    })

    return NormalizedRules(
        alphabet_size=len(tiles_sorted),
        color_size=len(colors_sorted),
        tile_int=tile_int,
        color_int=color_int,
        east_ok=east_ok,
        north_ok=north_ok,
        rules_hash=rules_hash,
    )
```

11.6 10.6 acs/input/cnf$_t emplates.py(Skeleton)$

```
from acs.core.types import NormalizedRules
from typing import Dict, Any

def build_cnf_dimacs(rules: NormalizedRules, L: int, placement: tuple[int,int],
                     boundary_mode: str, payload: Dict[str, Any]) -> str:
    # Skeleton: returns a DIMACS string.
    # Real implementation: exact-one constraints + adjacency + payload constraints.
    # Must be deterministic for same input.
```

```
    header = f"c CNF for L={L} placement={placement} boundary={boundary_mode} kind={
        payload.get('kind')}\n"
    header += "p cnf 1 1\n"
    header += "1 0\n"
    return header
```

11.7 10.7 acs/process/window$_{p}olicy.py$

```
from dataclasses import dataclass
from typing import List, Tuple

@dataclass(frozen=True)
class WindowPolicy:
    L_values: List[int]
    placements: List[Tuple[int,int]]
    boundary_modes: List[str] # "free" | "periodic" | "fixed_rim"
    interior_margin: int # >= 3*marker_radius recommended

def default_policy(marker_radius: int) -> WindowPolicy:
    m = max(3*marker_radius, 6)
    return WindowPolicy(
        L_values=[24, 32],
        placements=[(0,0), (5,7), (11,3)],
        boundary_modes=["free", "periodic"],
        interior_margin=m
    )
```

11.8 10.8 acs/process/obligations.py

```
from dataclasses import dataclass
from typing import Any, Dict, List
from acs.core.types import Certificate, NormalizedRules
from acs.process.window_policy import WindowPolicy

@dataclass
class SatQuery:
    name: str
    L: int
    placement: tuple[int,int]
    boundary_mode: str
    payload: Dict[str, Any]

@dataclass
class ObligationSet:
    sat_queries: List[SatQuery]
    finite_checks: List[Dict[str, Any]]

def generate_obligations(rules: NormalizedRules, cert: Certificate, policy: WindowPolicy)
     -> ObligationSet:
    sat_queries: List[SatQuery] = []
    finite_checks: List[Dict[str, Any]] = []
```

```
    # O1: marker soundness (misphased interior placements must be UNSAT)
    # + sanity: legal-phase marker occurrence should be SAT at least once
    for L in policy.L_values:
        for pl in policy.placements:
            for bm in policy.boundary_modes:
                sat_queries.append(SatQuery(
                    name="O1_marker_soundness",
                    L=L, placement=pl, boundary_mode=bm,
                    payload={"kind":"marker_soundness", "cert":cert, "margin":policy.
                        interior_margin}
                ))
                sat_queries.append(SatQuery(
                    name="O1_marker_sanity",
                    L=L, placement=pl, boundary_mode=bm,
                    payload={"kind":"marker_sanity", "cert":cert, "margin":policy.
                        interior_margin}
                ))
                sat_queries.append(SatQuery(
                    name="O2_unique_phase",
                    L=L, placement=pl, boundary_mode=bm,
                    payload={"kind":"unique_phase", "cert":cert}
                ))
                sat_queries.append(SatQuery(
                    name="O3b_prototype_coverage",
                    L=max(L, cert.k + 2), placement=pl, boundary_mode=bm,
                    payload={"kind":"prototype_coverage", "cert":cert}
                ))

    # O3: deflation consistency = finite checks by default
    finite_checks.append({"name":"O3_deflation_consistency", "kind":"prototype_legality",
         "cert":cert})

    # O4 optional
    if cert.invariant.type != "none":
        finite_checks.append({"name":"O4_invariant", "kind":"invariant_local", "cert":cert
            })

    return ObligationSet(sat_queries=sat_queries, finite_checks=finite_checks)
```

11.9 10.9 acs/output/sat$_{r}unner.py$

```
from dataclasses import dataclass
from acs.core.types import SatResult, TriState, SatCallMeta
from acs.core.hashing import sha256_str

@dataclass(frozen=True)
class SatConfig:
    solver: str = "cadical"
    solver_version: str = "unknown"
    seed: int = 0
    timeout_s: float = 10.0
    deterministic: bool = True
```

```python
class SatRunner:
    def __init__(self, cfg: SatConfig):
        self.cfg = cfg

    def solve(self, cnf_dimacs: str) -> SatResult:
        # Skeleton -- integrate a real solver later (PySAT or subprocess).
        cnf_hash = sha256_str(cnf_dimacs)
        meta = SatCallMeta(
            cnf_hash=cnf_hash,
            solver=self.cfg.solver,
            solver_version=self.cfg.solver_version,
            seed=self.cfg.seed,
            timeout_s=self.cfg.timeout_s,
        )
        return SatResult(status=TriState.UNKNOWN, sat=None, witness=None, meta=meta,
                         error="SatRunner not implemented (skeleton)")
```

11.10 10.10 acs/output/verifier.py

```python
import uuid
from acs.core.types import (TriState, Certificate, NormalizedRules, VerifierReport,
                            ObligationResult)
from acs.process.obligations import ObligationSet
from acs.input.cnf_templates import build_cnf_dimacs
from acs.output.sat_runner import SatRunner
from acs.core.hashing import hash_dict

REQUIRED_OBLIGATIONS = {
    "O1_marker_soundness",
    "O2_unique_phase",
    "O3_deflation_consistency",
    "O3b_prototype_coverage"
}

def _combine_results(obligs: dict[str, ObligationResult]) -> TriState:
    # HARD LOCK: PASS only if all required are PASS and none UNKNOWN
    for name in REQUIRED_OBLIGATIONS:
        if name not in obligs:
            return TriState.UNKNOWN
        if obligs[name].status == TriState.FAIL:
            return TriState.FAIL
        if obligs[name].status == TriState.UNKNOWN:
            return TriState.UNKNOWN

    # required all PASS; if any other FAIL => FAIL; if any UNKNOWN => UNKNOWN
    for r in obligs.values():
        if r.status == TriState.FAIL:
            return TriState.FAIL
        if r.status == TriState.UNKNOWN:
            return TriState.UNKNOWN
    return TriState.PASS
```

```python
def verify(rules: NormalizedRules, cert: Certificate, oblig_set: ObligationSet, sat:
    SatRunner) -> VerifierReport:
    run_id = str(uuid.uuid4())
    cert_hash = hash_dict({
        "rules_hash": cert.rules_hash,
        "k": cert.k,
        "markers": {"radius": cert.markers.radius, "patches": [p.grid for p in cert.
            markers.patches]},
        "deflation": {"protos": cert.deflation.macro_prototypes}
    })

    results: dict[str, ObligationResult] = {}
    sat_call_count: dict[str,int] = {}

    # SAT obligations
    for q in oblig_set.sat_queries:
        cnf = build_cnf_dimacs(rules, q.L, q.placement, q.boundary_mode, q.payload)
        sr = sat.solve(cnf)

        sat_call_count[q.name] = sat_call_count.get(q.name, 0) + 1
        if q.name not in results:
            results[q.name] = ObligationResult(name=q.name, status=TriState.PASS,
                sat_calls=0)

        results[q.name].sat_calls = sat_call_count[q.name]

        # NOTE: Proper mapping depends on payload semantics.
        # - For UNSAT-required claims (O1/O2/O3b): PASS iff UNSAT; FAIL iff SAT(witness);
            UNKNOWN otherwise.
        # - For SAT-required sanity (O1_marker_sanity): PASS iff SAT; FAIL iff UNSAT;
            UNKNOWN otherwise.
        # Skeleton runner yields UNKNOWN, so status becomes UNKNOWN.
        if sr.status == TriState.UNKNOWN:
            results[q.name].status = TriState.UNKNOWN
        else:
            results[q.name].status = TriState.UNKNOWN # placeholder until solver
                integrated

    # Finite checks
    for chk in oblig_set.finite_checks:
        name = chk["name"]
        if name not in results:
            results[name] = ObligationResult(name=name, status=TriState.PASS, checks=0)
        # Skeleton: mark PASS; real implementation must check prototype legality &
            boundary matches etc.
        results[name].checks += 1

    global_result = _combine_results(results)
    return VerifierReport(
        run_id=run_id,
        rules_hash=rules.rules_hash,
        cert_hash=cert_hash,
        result=global_result,
        obligations=results,
```

```
        stability={"note": "multi-window/multi-boundary policy enforced by ObligationSet"
            },
        environment={"solver": sat.cfg.solver, "solver_version": sat.cfg.solver_version, "
            seed": sat.cfg.seed}
    )
```

11.11 10.11 acs/process/proposer$_h$euristic.py

```
from acs.core.types import (Certificate, Markers, MarkerPatch, Deflation, Invariant)
from acs.core.hashing import sha256_str
import time

def propose_minimal(rules_hash: str, k: int) -> Certificate:
    # Minimal placeholder certificate -- for pipeline wiring only.
    markers = Markers(
        radius=2,
        patches=[MarkerPatch(id="M1", grid=[[0,0],[0,0]])],
        intended_phase=(0,0),
    )
    deflation = Deflation(
        macro_prototypes={"B0": [[0]*k for _ in range(k)]},
        macro_adjacency_east=[("B0","B0")],
        macro_adjacency_north=[("B0","B0")],
        iterable=True,
    )
    inv = Invariant(type="none", params={})
    return Certificate(
        cert_version="1.0",
        rules_hash=rules_hash,
        k=k,
        markers=markers,
        deflation=deflation,
        invariant=inv,
        provenance={"proposer":"heuristic_stub", "ts": time.time(), "id": sha256_str(
            rules_hash)[:16]}
    )
```

11.12 10.12 acs/process/cegis.py

```
from acs.core.types import NormalizedRules, Certificate, VerifierReport
from acs.process.window_policy import default_policy
from acs.process.obligations import generate_obligations
from acs.process.proposer_heuristic import propose_minimal
from acs.output.verifier import verify
from acs.output.sat_runner import SatRunner

def discover(rules: NormalizedRules, sat: SatRunner, k: int = 2, max_iters: int = 20) ->
    tuple[Certificate, VerifierReport]:
    # v1: only heuristic stub + verifier wiring
    cert = propose_minimal(rules.rules_hash, k=k)
    policy = default_policy(marker_radius=cert.markers.radius)
```

```
    obligs = generate_obligations(rules, cert, policy)
    report = verify(rules, cert, obligs, sat)
    # Real CEGIS: if FAIL -> use counterexample to refine markers/deflation
    return cert, report
```

11.13 10.13 acs/cli.py

```
import argparse
from acs.input.rulespec_parser import load_rulespec
from acs.input.canonicalize import canonicalize
from acs.output.sat_runner import SatRunner, SatConfig
from acs.process.cegis import discover

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--rules", required=True)
    ap.add_argument("--k", type=int, default=2)
    ap.add_argument("--timeout", type=float, default=10.0)
    args = ap.parse_args()

    spec = load_rulespec(args.rules)
    rules = canonicalize(spec)

    sat = SatRunner(SatConfig(timeout_s=args.timeout, seed=0))
    cert, report = discover(rules, sat, k=args.k)

    print("RESULT:", report.result)
    for name, r in report.obligations.items():
        print(name, r.status, "sat_calls=", r.sat_calls, "checks=", r.checks)

if __name__ == "__main__":
    main()
```

12 11. Notes on Completing the Skeleton (Implementation Checklist)

This document intentionally includes a runnable *wiring skeleton*. To make ACS functional:

1. Implement full CNF encoding for Wang tilings:
   - exact-one per cell, neighbor match constraints, boundary mode encodings.
2. Implement payload constraints:
   - O1 misphased marker placement (interior margin),
   - O1 sanity legal-phase marker placement,
   - O2 two-phase existence encoding,
   - O3b existence of legal $k \times k$ block not equal to any prototype.
3. Integrate a real solver backend:

- PySAT (python-native) or subprocess to CaDiCaL/Kissat/Glucose,
- deterministic config + CNF hashing already specified.

4. Implement O3 finite checks:
   - prototype legality (is each $k \times k$ block locally satisfiable),
   - boundary match between adjacent prototypes implies base-level compatibility.
5. Implement DATA stores and lineage enforcement (key by rules_hash).

13 References

References

[1] Robert Berger. The undecidability of the domino problem. *Memoirs of the American Mathematical Society*, No. 66, 1966.

[2] Raphael M. Robinson. Undecidability and nonperiodicity for tilings of the plane. *Inventiones Mathematicae*, 12(3):177–209, 1971.

[3] Douglas Lind and Brian Marcus. *An Introduction to Symbolic Dynamics and Coding*. Cambridge University Press, 1995.

[4] Lorenzo Sadun. *Topology of Tiling Spaces*. American Mathematical Society, 2008.

[5] Alessandro Abate et al. Counterexample guided inductive synthesis modulo theories. In *CAV*, 2018.